# Defence practice

20 QUESTIONS | 20 MINUTES | 15 SINGLE + 5 MULTIPLE

Try the paper before reading the key on page 7. Questions 1-15: choose one answer; 16-20: select all that apply. Vertex and original edge IDs start at 0. This is ungraded practice.

## 01 DSU operation states

CHOOSE ONE

Start with seven singleton components. After each union below, record [components, largest_size]. Which sequence is correct?

```
n = 7
unions = [(1,4), (4,6), (1,6), (0,2)]
```

A [[6,2], [5,3], [4,3], [3,3]]
B [[6,2], [5,3], [5,3], [4,3]]
C [[6,2], [5,2], [5,2], [4,2]]
D [[6,2], [5,3], [5,4], [4,4]]

## 02 Whole parent array

CHOOSE ONE

What is the entire parent array after this path-compression loop?

```
int[] parent = {0, 0, 1, 2, 4, 4};
int root = 0, x = 3;
for (int u = x; u != root; ) {
    int next = parent[u];
    parent[u] = root;
    u = next;
}
```

A [0,0,1,0,4,4]
B [0,0,0,2,4,4]
C [0,0,0,0,0,0]
D [0,0,0,0,4,4]

## 03 Original edge IDs

CHOOSE ONE

For this directed graph, which original edge IDs leave vertex 2? Keep input order and include self-loops.

```
n = 4; vertex = 2
edges = [[2,0,7], [0,1,3], [2,2,-1],
         [1,2,4], [2,3,2]]
```

A [0,2,4]
B [0,3,4]
C [0,4]
D [1,2,3]

# Defence practice

## 04 Kruskal trace

CHOOSE ONE

Run Kruskal in increasing weight order. Which original edge IDs are accepted, in acceptance order?

```
n = 5
edges = [[0,1,5], [1,2,1], [0,2,3],
         [3,4,-2], [2,2,-7]]
```

**A** [4,3,1,2]
**B** [3,1,0]
**C** [3,1,2]
**D** [1,2,3]

## 05 Prim heap candidates

CHOOSE ONE

A lazy Prim implementation has visited vertices {0,2}. Which edge is accepted next after discarding stale heap entries? Each tuple is (weight, from, to, edge_id).

```
heap candidates:
(6,0,1,0), (2,2,1,1), (1,2,0,2), (4,2,3,3)
```

**A** edge_id 2
**B** edge_id 1
**C** edge_id 3
**D** edge_id 0

## 06 DFS finish sequence

CHOOSE ONE

Start unvisited roots in increasing vertex order. Scan outgoing neighbours in input-row order. In which order do vertices finish?

```
n = 5
edges = [[0,2], [0,1], [2,3], [1,3]]
```

**A** [3,2,1,0,4]
**B** [0,2,3,1,4]
**C** [3,1,2,0,4]
**D** [4,0,1,2,3]

## 07 SCC membership

CHOOSE ONE

Which component-label array represents exactly the strongly connected components? Labels are arbitrary nonnegative integers.

```
n = 6
edges = [[0,2], [2,0], [2,1], [1,3],
         [3,1], [3,4], [4,4]]
```

**A** [8,8,8,8,9,12]
**B** [8,4,8,9,9,12]
**C** [8,4,8,4,12,12]
**D** [8,4,8,4,9,12]

# Defence practice

## 08 Topological order

CHOOSE ONE

Which sequence is a valid topological order of all five vertices?

```
edges = [[0,3], [1,3], [1,2], [2,4]]
```

A [0,3,1,2,4]
B [1,0,4,2,3]
C [1,2,0,4,3]
D [2,1,0,3,4]

## 09 DAG shortest distances

CHOOSE ONE

What is the distance array? The graph is directed; null denotes an unreachable vertex.

```
n = 6; source = 1
edges = [[1,0,4], [1,2,7], [0,2,-6],
         [2,3,3], [0,3,5], [4,5,-8]]
```

A [4,0,7,9,null,null]
B [4,0,-2,1,null,null]
C [4,0,-2,1,0,-8]
D [0,4,-6,-3,null,null]

## 10 Route reconstruction

CHOOSE ONE

Using these original edge IDs, reconstruct the route from source 1 to target 3. Which pair gives the vertex route and its edge IDs?

```
edges = [[1,0,4], [1,2,7], [0,2,-6],
         [2,3,3], [0,3,5], [4,5,-8]]
parent_edge = [0,-1,2,3,-1,-1]
```

A vertices [1,0,2,3]; edge IDs [0,2,3]
B vertices [1,2,3]; edge IDs [1,3]
C vertices [3,2,0,1]; edge IDs [3,2,0]
D vertices [1,0,2,3]; edge IDs [1,0,2]

## 11 Minimum spanning forest

CHOOSE ONE

Which triple is (minimum cost, component count, selected-edge count)? The graph is undirected.

```
n = 6
edges = [[0,1,-3], [1,2,4], [0,2,2],
         [3,4,6], [3,4,1], [5,5,-20]]
```

A (-20,3,3)
B (0,2,4)
C (2,3,3)
D (0,3,3)

# Defence practice

## 12 Condensation graph

CHOOSE ONE

For the given valid SCC labels, which condensation array is correct? Pair order is irrelevant; duplicate pairs must be omitted.

```
component = [2,2,8,8,5]
edges = [[0,1], [1,2], [0,3], [2,3],
         [3,4], [2,4], [4,4]]
```

**A** [[2,8],[8,5],[5,5]]
**B** [[2,8],[8,5]]
**C** [[2,8],[2,8],[8,5],[8,5]]
**D** [[8,2],[5,8]]

## 13 Union by size

CHOOSE ONE

Call union(1,4), attaching the smaller root below the larger root. Initially there are three components. Which triple is (parent[0], size[2], components) afterward? Sizes at non-roots are ignored.

```
parent = [0,0,2,2,2,5]
size   = [2,1,3,1,1,1]
```

**A** (0,5,2)
**B** (2,3,2)
**C** (2,5,2)
**D** (2,5,1)

## 14 Cycles outside source reachability

CHOOSE ONE

Which result satisfies the Assignment 3 DAG contract?

```
n = 5; source = 0
edges = [[0,1,2], [2,3,1], [3,2,1]]
```

**A** acyclic=false; distance=[]; parent_edge=[]
**B** acyclic=true; distance=[0,2,null,null,null]; parent_edge=[-1,0,-1,-1,-1]
**C** acyclic=false; distance=[0,2,null,null,null]; parent_edge=[-1,0,-1,-1,-1]
**D** acyclic=true; distance=[]; parent_edge=[]

## 15 Benchmark medians

CHOOSE ONE

Both methods were timed on the same graph with the same timing boundary. Which pair gives (Kruskal median, Prim median), in milliseconds?

```
Kruskal: [9,4,6] ms
Prim:    [5,8,7] ms
```

**A** (4,5)
**B** (9,8)
**C** (6.33,6.67)
**D** (6,7)

# Defence practice

## 16 Alternative optimal forests

SELECT ALL THAT APPLY

Select all edge-ID sets that are minimum spanning forests. IDs inside a set are unordered. The graph is undirected.

```
n = 5
edges = [[0,1,2], [1,2,2], [0,2,2],
         [2,3,5], [1,3,9]]
```

A [0,1,3]
B [0,1,4]
C [0,2,3]
D [0,1,2]
E [1,2,3]

## 17 DFS active edges

SELECT ALL THAT APPLY

Use three-state DFS. Start roots in increasing vertex order and scan neighbours in input-row order. Select all edges that lead to an active vertex when examined.

```
n = 5
edges = [[0,1], [1,2], [2,0], [1,3],
         [3,1], [3,4], [4,4]]
```

A ID 0: 0 -> 1
B ID 2: 2 -> 0
C ID 4: 3 -> 1
D ID 5: 3 -> 4
E ID 6: 4 -> 4

## 18 Topological witnesses

SELECT ALL THAT APPLY

Select all valid topological orders. Each must contain every vertex exactly once.

```
n = 4
edges = [[0,2], [1,2], [1,3]]
```

A [0,2,1,3]
B [0,1,2,3]
C [1,2,0,3]
D [3,1,0,2]
E [1,0,3,2]

# Defence practice

## 19 Shortest-path parent witnesses

SELECT ALL THAT APPLY

Select all valid parent_edge arrays for shortest paths from source 0. Entries store original incoming edge IDs.

```
n = 4; source = 0
edges = [[0,1,3], [0,1,3], [1,2,-5],
         [0,2,0], [2,3,4], [1,3,2]]
```

**A** [-1,0,2,4]
**B** [-1,0,3,4]
**C** [-1,2,2,4]
**D** [-1,1,2,4]
**E** [-1,0,2,5]

## 20 Test cases for a serialization defect

SELECT ALL THAT APPLY

A DAG solver computes correct finite distances but serializes every distance equal to -1 as null. Select all requests that expose this defect. All requests have n=3 and source=0; only edges are shown.

**A** [[0,1,2],[1,2,-3]]
**B** [[0,1,4],[1,2,-2]]
**C** [[0,1,-1]]
**D** [[1,2,-1]]
**E** [[0,1,-2],[1,2,1]]

# Answers & explanations

## 01 DSU operation states — B

The first two unions create {1,4,6}. The third joins vertices already in that component, so it changes neither count nor size. The last creates {0,2}. The remaining singleton vertices are 3 and 5.

## 02 Whole parent array — D

The old path is 3 -> 2 -> 1 -> 0. Saving next before assignment lets the loop visit the whole old path. Vertices 1, 2 and 3 point to root 0; the separate component {4,5} is unchanged.

## 03 Original edge IDs — A

An outgoing edge has 2 in its first column. Rows 0, 2 and 4 satisfy this test. Row 3 enters vertex 2; row 2 is a self-loop and is included once. IDs refer to original rows.

## 04 Kruskal trace — C

The sorted order is 4, 3, 1, 2, 0. Edge 4 is a self-loop and is rejected. Edges 3, 1 and 2 connect different components; edge 0 then closes a cycle. Cost is -2 + 1 + 3 = 2, with two components.

## 05 Prim heap candidates — B

The lightest entry, edge 2, ends at already visited vertex 0 and is stale. The next lightest edge reaching an unvisited vertex is edge 1, with weight 2 and destination 1.

## 06 DFS finish sequence — A

DFS from 0 enters 2, then 3. Vertex 3 finishes before 2. Next, DFS visits 1, whose neighbour 3 is already finished. Finally 0 finishes. The outer loop later visits isolated vertex 4. This is finish order, before reversal.

## 07 SCC membership — D

Mutual reachability gives {0,2}, {1,3}, {4}, {5}. The edges from one group to the next do not provide a return path, so they do not merge the groups. The self-loop at 4 does not connect it to another vertex.

# Answers & explanations

## 08 Topological order — C

In [1,2,0,4,3], every edge goes from an earlier vertex to a later one. The other choices violate 1 -> 3, 2 -> 4 or 1 -> 2, respectively. The assignment accepts any valid order, not only one DFS output.

## 09 DAG shortest distances — B

Vertex 0 costs 4. Reaching 2 through 0 costs 4 - 6 = -2, improving on 7. Reaching 3 through 2 costs -2 + 3 = 1, improving on 9. Vertices 4 and 5 are unreachable; their edge must not be relaxed from an invented finite distance.

## 10 Route reconstruction — A

From target 3, parent edge 3 leads back to 2; edge 2 leads back to 0; edge 0 leads back to source 1. Reverse the collected sequence. The selected edge weights sum to 4 - 6 + 3 = 1. Parent entries are edge IDs, not predecessor vertices.

## 11 Minimum spanning forest — D

For {0,1,2}, select weights -3 and 2, giving -1. For {3,4}, select the cheaper parallel edge of weight 1. Vertex 5 is isolated for connectivity purposes; its loop cannot enter a forest. Total cost is 0, with three components and n-c = 3 edges.

## 12 Condensation graph — B

Remove edges whose endpoints have equal labels. Map the other edges to label pairs and deduplicate: 1 -> 2 and 0 -> 3 both become (2,8); 3 -> 4 and 2 -> 4 both become (8,5). Keep the original directions.

## 13 Union by size — C

find(1)=0 and find(4)=2. Root 0 has size 2 and root 2 has size 3, so parent[0] becomes 2. Add the component sizes at root 2 to obtain 5. One successful merge decreases the component count from 3 to 2.

## 14 Cycles outside source reachability — A

Vertices 2 and 3 form a directed cycle even though the source cannot reach them. The required cycle check covers the entire graph. Any directed cycle requires acyclic=false and both arrays empty.

# Answers & explanations

**15** **Benchmark medians** **D**

Sort each sample: [4,6,9] and [5,7,8]. The middle values are 6 and 7. Kruskal has the lower median for this measured graph and environment; that observation alone does not establish which method is faster on all graphs.

**16** **Alternative optimal forests** **A, C, E**

Vertex 4 is isolated. Any two edges from the equal-weight triangle connect {0,1,2} for cost 4. Add edge 3 to reach vertex 3 for total cost 9. Using edge 4 costs more; using all three triangle edges creates a cycle and leaves vertex 3 disconnected.

**17** **DFS active edges** **B, C, E**

DFS reaches 0 -> 1 -> 2, where edge 2 points back to active 0. After 2 finishes, DFS enters 3; edge 4 points to active 1. At vertex 4, the self-loop points to active 4. Edges 0 and 5 discover unvisited vertices, so they are tree edges.

**18** **Topological witnesses** **B, E**

Both 0 and 1 must precede 2, and 1 must precede 3. The second and fifth choices meet every condition. The first places 2 before 1; the third places 2 before 0; the fourth places 3 before 1.

**19** **Shortest-path parent witnesses** **A, D**

Distances are [0,3,-2,2]. Either parallel edge 0 or 1 may reach vertex 1. Edge 2 is needed to reach 2 at cost -2, and edge 4 reaches 3 at cost 2. Edge 3 gives the wrong distance to 2; edge 2 ends at 2, not 1; edge 5 reaches 3 at cost 5.

**20** **Test cases for a serialization defect** **A, C, E**

The first and fifth requests produce distance -1 at vertex 2; the third produces -1 at vertex 1. The second has only 0, 4 and 2 as finite distances. In the fourth, vertices 1 and 2 are unreachable, so null is already the correct output for them.

**SELF-CHECK SCORING**

Each question is worth 1 raw mark. For questions 1-15, a correct answer earns 1; an incorrect or blank answer earns 0. For questions 16-20, each selected correct option earns 1/k and each selected incorrect option deducts 1/(2k), where k is the number of correct options. Add the weights; the minimum score is 0. Blank answers earn 0.

Two correct options: +0.50 and -0.25. Three correct options: +1/3 and -1/6. One correct plus one incorrect gives 0.25 or about 0.17. Selecting every option gives 0.25 or about 0.67. Full credit requires every correct option and no incorrect options.

**Practice score out of 50 = total raw marks / 20 x 50.**

Assignment specification / Interactive lesson